Semester Project Document

From SemSteDiff → AgentSemSteCom → Our Proposed RACS (Retrieval-Augmented Coverless Steganography)

0. The Core Problem (Context for Everything Below)

We want to send a **private/secret image** (e.g., a medical scan, a personal photo) over a wireless network without an eavesdropper being able to see it — even if they intercept the transmission.

Two classic approaches:

Encryption — scrambles the image into gibberish. Problem: gibberish is *obviously* suspicious, inviting attackers to try harder.
Steganography — hides the secret image inside an innocent-looking "cover" image, so eavesdroppers don't even suspect anything is hidden.

Our entire project lives in the **steganography** space, specifically a modern flavor of it called **coverless semantic steganography**, where AI generates the cover image instead of using a real pre-existing photo as a cover.

1. The Earlier Method: SemSteDiff (Prior Work, ~2025)

The Idea

SemSteDiff was one of the first systems to do **coverless** steganography using diffusion models (the same tech behind Stable Diffusion / DALL-E).

How It Worked

The secret image is converted into a description using two things:
A **public semantic key** — a general text description (e.g., "*a horse standing in a field*") that's safe to share openly.
A **private semantic key** — a more specific/sensitive text description (e.g., "*a sheep standing in a field next to a wooden fence*") that must stay secret.
A diffusion model uses both keys to generate a stego image — an image that looks like an ordinary photo but secretly encodes the private information.
To recover the secret image, the receiver needs the **private semantic key** as a kind of password.

The Problem With This Method

The private semantic key is **content-related** — it's literally a text description of what's in the secret image. This creates a serious weakness:

If an intelligent eavesdropper knows the *context* (e.g., "this is a hospital's data channel" or "this patient has a history of fractures"), they can **guess** the private semantic key using their own AI reasoning, without ever needing to intercept it directly.

Real-world example the researchers use: In a hospital's wireless network, an eavesdropper aware of a patient's medical history might guess the private key is something like "*fracture*" or "*tumor*" — and successfully reconstruct the private diagnostic image, defeating the entire steganography system.

Summary of SemSteDiff's Weakness

Weakness	Explanation
Guessable secret key	The "password" is a meaningful text phrase tied to the image content
Repeated exposure risk	A new private key is needed for every image, increasing leak opportunities
No agent/reasoning	The system is a fixed, static pipeline — it can't adapt to context or defend itself

2. The Base Paper's Method: AgentSemSteCom (2026)

This is the paper we are directly building on. It fixes SemSteDiff's core weakness.

The Key Insight

Stop using a guessable, content-based secret key. Use a meaningless, user-defined number instead — a "digital token."

A digital token (e.g., `9000`, `76576`) carries **no semantic meaning** on its own. There's nothing for an eavesdropper's AI to "reason about" or guess based on context — it's just a random number, shared securely once, out of band (like exchanging a Wi-Fi password in person, not over the network you're trying to protect).

The Full Pipeline (5 Modules)

Module 1 — Semantic Extraction An LLM (AI language model) looks at the secret image and generates:

A **public semantic key** — a plain text description (e.g., "*an elderly woman smiling outdoors*"). This is safe to make public because it reveals nothing sensitive.

An **implicit feature** — a structural map (e.g., a segmentation map for general objects, or a facial skeleton/pose map for faces). This preserves the *shape/layout* of the secret image without revealing *what* it actually shows.

Module 2 — Digital Token Controlled Reference Image Generation

The digital token is converted into a random seed (using a hash function — think of it as a formula that turns any number into a unique, reproducible "starting point").
This seed initializes a diffusion model (using ControlNet to respect the structural map from Module 1), which **generates a brand-new fake "reference image"** — e.g., a realistic-looking face of "someone else."
Key property: The same token always produces the exact same reference image. Different tokens produce completely different reference images from the very same secret image.

Module 3 — Conditional Diffusion Model-Based Coverless Steganography (the actual hiding)

The secret image is mathematically "dissolved" into pure noise using a special **reversible** diffusion technique called **EDICT** (Exact Diffusion Inversion via Coupled Transformations). Unlike standard diffusion (which loses information when reversed, due to rounding/approximation errors), EDICT runs **two coupled parallel chains** that mathematically guarantee the process can be perfectly undone.
This noise is then **scrambled further** using the digital token — a random binary mask flips the sign of specific values in the noise, controlled by the token (like shuffling specific cards in a deck based on a secret pattern).
The scrambled noise is then guided back into a real-looking image — the **stego image** — using the public semantic key and reference image (from Module 2) as steering signals.
This stego image is what actually gets transmitted over the wireless channel.

Module 4 — JSCC-Based Semantic Codec (Wireless Transmission)

The stego image is encoded into a compact "semantic feature" signal and sent through a noisy wireless channel (modeled with realistic radio-noise, called AWGN — Additive White Gaussian Noise).
The receiver decodes this signal back into an approximate copy of the stego image.

Module 5 — Recovery + Optional Enhancement

The receiver, using the *same* digital token and public key, regenerates the *same* reference image (Module 2, run again), then reverses the EDICT process (Module 3, run backward) to recover the original secret image.
If the recovered image is a bit noisy or low-quality (due to wireless channel distortion), an optional AI enhancement tool (e.g., a face-restoration model) cleans it up.

Why It's Better Than SemSteDiff

	SemSteDiff	AgentSemSteCom
Secret key type	Meaningful text (guessable)	Meaningless number (not guessable)
Needs original cover image?	No (coverless)	No (coverless)
Reversibility	Approximate (lossy)	Exact (via EDICT)
Adaptivity	Fixed pipeline	Agentic AI adapts strategy per image/task
Result (from their experiments)	Baseline	+2.69 dB PSNR, +8.88% SSIM, -43.75% MSE, -5.1% LPIPS over SemSteDiff

The Remaining Weakness (Our Opportunity)

Look closely at **Module 2**: every single time, for every single secret image, the system runs a **full diffusion model from pure noise** to generate a brand-new fake reference image. This is:

Computationally expensive (many neural network passes — 50 diffusion steps)
Slow (not usable for real-time or low-power devices — a real concern for 6G edge/IoT devices, which is literally the target use-case of "intellicise" networks)

The authors even admit this in their own conclusion, listing "step-efficient denoising" and "reducing computational constraints" as unsolved future work.

3. Our Proposed Solution: RACS (Retrieval-Augmented Coverless Steganography)

The Core Idea (One Sentence)

Instead of generating a brand-new fake reference image from scratch every time, search a huge existing image database and retrieve one that's already similar — then lightly touch it up. Use the digital token to control *which* image gets picked, not *what noise* gets generated.

Simple Analogy

AgentSemSteCom's way: Every time you need a disguise, you hire an artist to paint a brand-new face from imagination, from nothing. Slow, expensive, done from scratch every time.
Our way: You already own a giant photo album with millions of real faces. You just flip to a specific page (chosen by a secret number) and grab that photo instantly — no painting needed.

Why This Is a Different *Architecture*, Not Just a Faster Version

This isn't "use a smaller/faster diffusion model" (which would still be generating from scratch, just quicker). We are removing the generation step almost entirely for the reference image, and replacing it with **search**. This is inspired by a technique from the language-AI world called **Retrieval-Augmented Generation (RAG)** — commonly used to make chatbots faster and more accurate by having them "look things up" instead of "making things up" — but this exact idea has not yet been applied to coverless image steganography.

The Full Pipeline (Our Version)

Setup Phase (done once, ahead of time, offline)

Take the **LAION dataset** (a massive, publicly available collection of billions of image-text pairs — the same dataset used to train models like Stable Diffusion).
Convert every image into a compact numerical "fingerprint" (an **embedding**) using a pretrained model called **CLIP**. Images that look/mean similar things get mathematically similar fingerprints.
Store all these fingerprints in a fast searchable index — a **vector database** (built using a tool like FAISS) — so we can instantly find "the most similar images" to any given description.

Sending a Secret Image (Modified Pipeline)

Module 1 (same as before): An LLM writes a public text description of the secret image (e.g., "*elderly woman, gray hair, smiling, outdoors*").

Module 2 (OUR CHANGE — Retrieval Instead of Generation):

Convert the text description into a CLIP embedding.
Search the LAION vector database for the top-K most similar real images (e.g., top 100 closest matches).
The digital token now decides which of these top matches gets selected (e.g., token `9000` → pick match #47; token `76576` → pick match #12). This replaces "token controls random noise" with "token controls which real image gets chosen" — a cleaner, more interpretable form of the same idea.
Apply a **light editing step** to this retrieved real image (a small, fast image-to-image adjustment — not a full 50-step generation) to nudge it closer to the secret image's structure/layout.

Module 3 (same idea as before): Hide the secret image inside this retrieved+edited reference image using the same invertible (EDICT-style) hiding process.

Module 4 (same as before): Transmit over the noisy wireless channel (kept as a simple backdrop simulation, not a research focus).

Module 5 (same as before): The receiver, using the same token, performs the same search + retrieval + light edit, then reverses the hiding process to recover the secret image.

What Changes vs. What Stays the Same

Component	AgentSemSteCom	Our RACS
Cover/reference image source	Generated from noise (diffusion, 50 steps)	Retrieved from existing database (near-instant)
What the token controls	Random noise seed	Which image (out of many) gets selected
Speed of reference creation	Slow (full diffusion run)	Fast (search + light edit)
Hiding mechanism (EDICT-based)	Same	Kept the same (not our focus)
Wireless transmission (JSCC)	Same	Kept the same (backdrop, not our focus)
Security guarantee	"Guess the right noise seed"	"Guess which of potentially millions of images was picked"

Why We Believe This Is Secure

The retrieved image itself is not meant to be secret — it just needs to look like an ordinary, plausible photo (same goal as the generated version).
What matters is: (1) the secret image hidden inside it stays protected, and (2) the exact image chosen (out of a huge database) is hard to guess without the token — this creates a very large "search space" for any attacker.
We will test this rigorously using the same kind of evaluation as the original paper (see below).

What We Will Measure (Evaluation Plan)

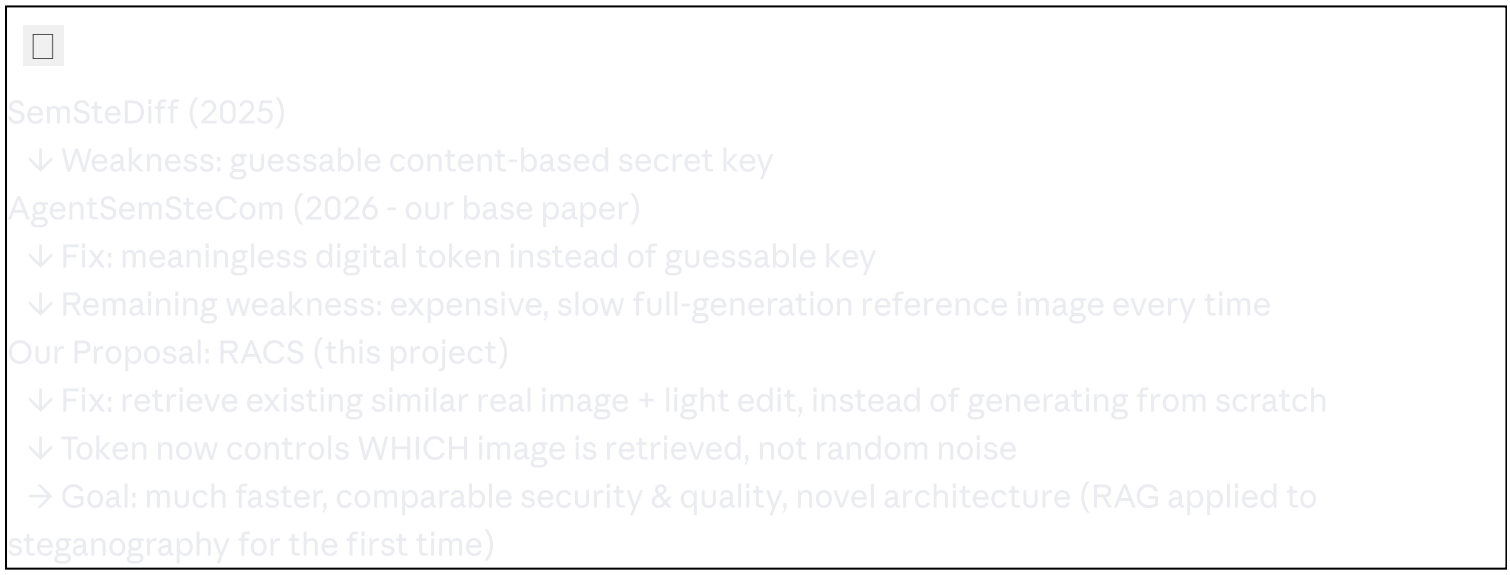
Metric Category	Specific Metrics	Purpose
Recovery Quality	PSNR, SSIM, MSE, LPIPS	How close is the recovered secret image to the original? (directly comparable to base paper's numbers)
Speed (our headline result)	Inference time per image, GPU memory used	How much faster is retrieval+light-edit vs. full diffusion generation?

Metric Category	Specific Metrics	Purpose
Security	Steganalysis detection accuracy (e.g., using a detector like XuNet)	Can automated tools tell stego images apart from normal images? (want close to 50%, i.e., random guessing)
Security	PSNR gap between legitimate receiver (correct token) vs. eavesdropper (wrong token)	Bigger gap = stronger security

Fallback Plan (If Retrieval Quality Is Too Weak)

If pure retrieval + light editing doesn't produce good enough structural matches, we fall back to a **hybrid mode**: use retrieval for most images where a good match exists in the database, and only fall back to full generation for rare/hard cases with no good match. This "graceful degradation" behavior is itself a small additional contribution.

4. Summary Timeline of Ideas



5. One-Paragraph Summary (For Quick Explanation to Anyone)

Sending private images over wireless networks is risky because eavesdroppers can intercept them. Steganography hides secret images inside innocent-looking "cover" images. Earlier systems used a text-based secret password to control this hiding, but smart AI eavesdroppers could guess that password from context. The paper we're building on fixed this by using a meaningless secret number (a "digital token") instead — but its process for creating the cover image is slow, because it

generates a brand-new fake image from scratch every single time using a heavy AI model. Our project proposes replacing that expensive generation step with **retrieval**: instead of drawing a new fake image, we search a massive existing image database (LAION) and pick out an already-similar real image, then lightly touch it up. The secret token now decides *which* image gets picked, instead of controlling random noise. This makes the whole system much faster while aiming to keep the same level of security and image-recovery quality — and it's the first time this "search instead of generate" idea (borrowed from chatbot research, called Retrieval-Augmented Generation) has been applied to this kind of image steganography problem.